

Comprehensive Preprocessing Methodology

Phase 1: Raw Data Extraction & Label Parsing

The dataset natively exists as 100,000 individual `.txt` files grouped in complex directory trees. To prepare this data for machine learning, Python scripts were used to parse the file structure:

Binary Labels: Extracted based on directory paths (files in `/pos` mapped to 1, `/neg` mapped to 0). **Continuous Regression Labels:** The exact 1-10 rating score was extracted directly from the text file metadata using string manipulation to parse the `[id]_[rating].txt` format. **DataFrame Construction:** The raw text and corresponding labels were loaded into tabular Pandas DataFrames for batch processing.

Phase 2: Core Text Cleaning Pipeline

Web-scraped natural language is notoriously noisy. Over 50% of the raw reviews contained structural HTML artifacts. A universal pipeline was applied across the codebase:

HTML Stripping: Regular Expressions (regex) `re.sub(r'<[^>]*>', ' ', text)` were utilized to permanently remove

tags and rogue HTML elements. **Whitespace Normalization:** Multiple spaces and tabs were collapsed into single spaces to ensure token consistency. **Lowercasing:** All text was converted to lowercase to collapse vocabulary variants (e.g., 'Movie' and 'movie' become mathematically identical). **Punctuation Filtering (EDA specific):** For WordCloud generation and frequency analysis, special characters were aggressively stripped using the regex `['^a-zA-Z\s']`.

Phase 3: Meta-Feature Extraction

Prior to destructive cleaning steps like lowercasing, meta-features were extracted for Exploratory Data Analysis (EDA):

String Lengths: The raw character count of each review was tracked. **Punctuation & Capitalization Counts:** The frequency of exclamation marks (!) and entirely capitalized words (e.g., `AWFUL`) were counted using regex `\b[A-Z]{2,}\b` to correlate shouting behavior with 1-10 rating scores.

Phase 4: Vectorization & Tokenization

Machine learning models cannot read raw text. Following the cleaning pipeline, the text was transformed into highly specific numerical representations tailored to each algorithm's architecture:

TF-IDF Vectorization (Logistic Regression, Naive Bayes, SVD): Text was tokenized into unigrams and bigrams, limited to the top 10,000 most frequent features. Term Frequency-Inverse Document Frequency (TF-IDF) scoring built a highly sparse numerical matrix, weighting words by their relative uniqueness across the corpus. **Integer Tokenization**

& Padding (LSTM): Text was split into tokens and passed through a custom `Word2Idx` dictionary builder. Each word was mapped to a unique integer. Sequences were subsequently padded with zeros or truncated to a fixed sequence length of 200 tokens to ensure uniform matrix dimensions for the Neural Network.

Sub-word Tokenization (BERT): Deep learning transformers utilize dynamic vocabularies. We utilized the pre-trained `bert-base-uncased` tokenizer, applying WordPiece sub-word tokenization to handle unknown words. Sentences were padded to a length of 512 tokens, and Attention Masks were generated to instruct the Transformer on which tokens represent valid words versus padding integers.